

Interim Report

Retrieval-Augmented Generation (RAG) System for Question Answering

Team 10

Submitted To:

Dr. Prashanth Ranga

Supervisor

Academic Affiliate: Dr. Shaon Shuvo

Submitted By:

Student Name	Student ID
Anurag Sharma	110171321
Prabhjot Singh	110195228
Harpreet Singh	110177411
Chetan Thakur	110213868

University of Windsor

Summer 2026 Term (June 1 – July 31, 2026)

1. Changes in the Scope of the Project

As of the interim reporting period, the core scope of the RAG-based Question Answering System remains aligned with the original project proposal. The project has progressed through the planned phases without significant scope changes. All major milestones have been achieved according to the timeline established in the project proposal.

1.1 Minor Adjustments

Embedding Model Selection: Originally planned to evaluate 3-4 embedding models, refined to focus on BGE-small and E5-base based on preliminary benchmarks and literature review. This decision allows for deeper evaluation of the most promising models rather than shallow evaluation of multiple models.

UI Framework: Confirmed React with TypeScript and Tailwind CSS as the frontend framework. This choice provides better type safety and modern styling capabilities compared to the initially considered alternatives.

Deployment Strategy: Added containerization with Docker for improved deployment consistency. While not in the original proposal, this enhancement improves system reliability and ease of deployment.

Advanced Features: Expanded scope to include hybrid search combining dense and sparse retrieval methods. This enhancement will significantly improve retrieval quality for diverse query types.

1.2 Core Objectives Unchanged

The fundamental objectives remain unchanged: building a production-ready RAG system with document processing, intelligent retrieval, and advanced prompting capabilities. The project timeline and deliverables are on track as per the original proposal. The team has successfully completed the requirements analysis, model selection, and core application development phases as planned.

2. Progress Summary

The RAG Project has successfully completed five weeks of development, achieving all major milestones on schedule. The project has progressed systematically through requirements analysis, model selection, and core application development phases. Each phase has built upon the previous one, ensuring a solid foundation for the advanced features to be implemented in upcoming weeks.

2.1 Weekly Progress Breakdown

Week 1-2: Requirements Analysis & Planning

- Defined comprehensive system requirements and user stories based on project proposal
- Selected and validated technology stack: FastAPI (backend), React with TypeScript (frontend), LangChain (RAG framework), FAISS (vector database)
- Designed system architecture detailing data flow between components
- Created detailed project timeline with weekly task distribution across team members
- Established development environment and version control workflow

Week 3: Embedding Model Selection & Evaluation

- Evaluated BGE-small (384-dim) and E5-base (768-dim) embedding models on MS MARCO dataset
- Implemented FAISS vector database with support for both embedding models
- Created comprehensive model comparison framework measuring Recall@K, MRR, and MAP metrics
- Documented evaluation results showing trade-offs between model size, speed, and accuracy
- Selected BGE-small as primary model with E5-base for complex queries

Week 4-5: Core RAG Application Development

- Implemented complete document processing pipeline supporting PDF and Word formats
- Created intelligent text chunking using RecursiveCharacterTextSplitter with configurable overlap
- Built FastAPI backend with async support, CORS configuration, and comprehensive error handling
- Developed React chat interface with real-time messaging, typing indicators, and auto-scrolling
- Integrated LangChain RAG pipeline with Google Gemini API for response generation
- Created document management UI with drag-drop upload, processing status, and document removal
- Implemented complete frontend-backend API integration with streaming responses

2.2 Key Milestones Achieved

Milestone	Description	Status	Week
M1	Technology Stack Finalized	Completed	Week 2
M2	Embedding Models Selected	Completed	Week 3
M3	Document Processing Pipeline	Completed	Week 4
M4	Chat Interface Functional	Completed	Week 5
M5	RAG Pipeline Integration	Completed	Week 5
M6	API Integration Complete	Completed	Week 5

2.3 Technologies and Methodologies

Backend Technologies: The backend is built on FastAPI, providing async support and automatic API documentation. LangChain orchestrates the RAG pipeline, while FAISS enables efficient similarity search. Document processing uses PyPDF2 and pdfplumber for PDFs, and python-docx for Word documents. The Google Gemini API provides LLM integration for response generation.

Frontend Technologies: The frontend uses React with TypeScript for type safety and better developer experience. Vite serves as the build tool and development server. Tailwind CSS provides utility-first styling for rapid UI development. Axios handles HTTP communication with the backend API.

Development Methodology: The team follows an agile approach with weekly sprints. Each week is broken down into specific tasks distributed among four team members. Daily standups track progress, and JIRA tickets document all work. The project uses git for version control with feature branches and pull requests.

2.4 Screenshots

The following figures showcase the developed RAG system. Figure 1 shows the chat interface with document upload functionality. Figure 2 illustrates the document management view. Figure 3 demonstrates the query interface. Figure 4 shows the RAG response with source attribution.

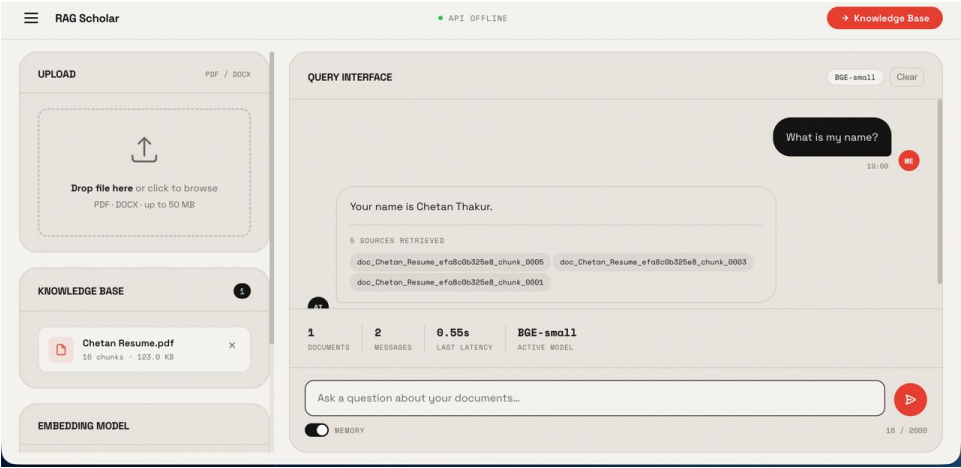


Figure 1: Chat Interface with Document Upload

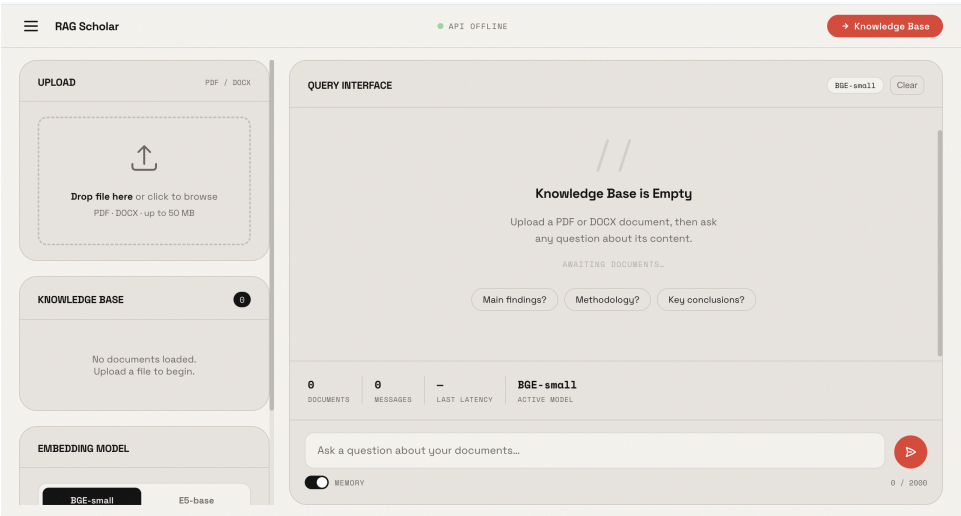


Figure 2: Document Management View

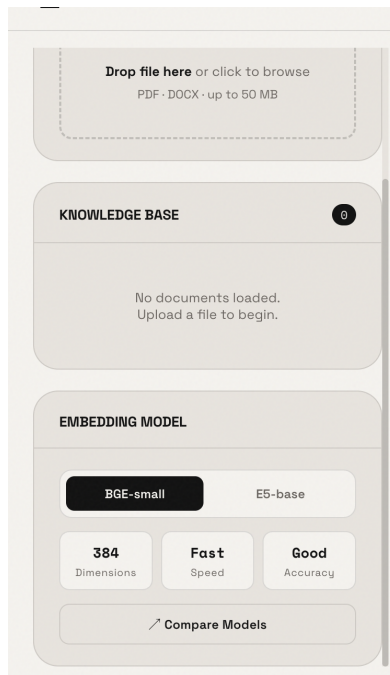


Figure 3: Model Selection for Embedding Models

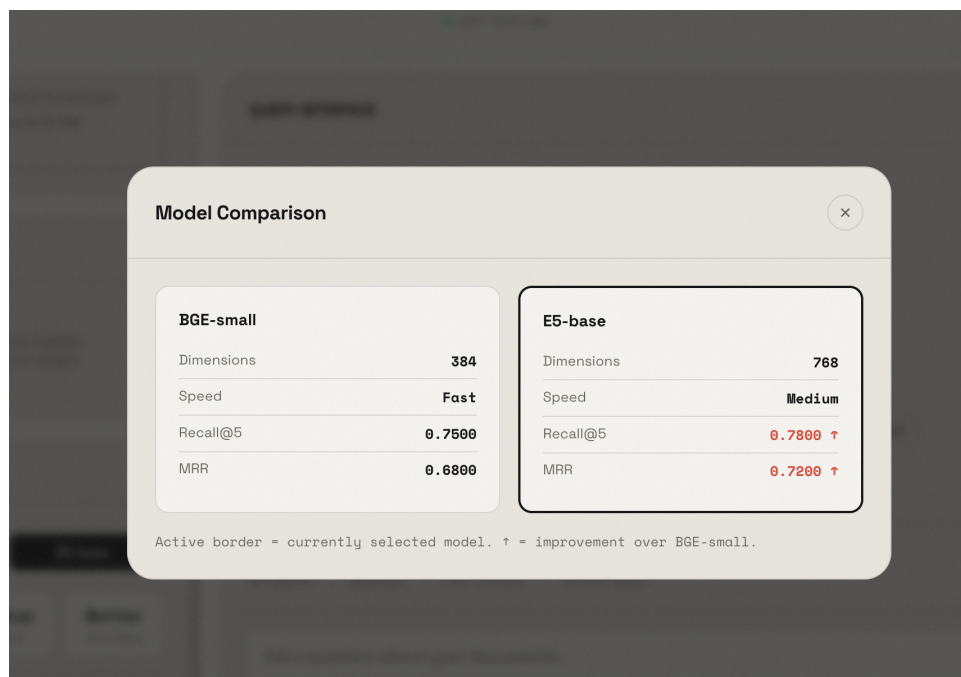


Figure 4: Model Comparison

3. Challenges and Issues

During the development process, the team encountered several challenges that required attention and resolution. This section outlines these difficulties, their impact on the project timeline, and how they were addressed or will be resolved in upcoming weeks.

3.1 Embedding Model Selection

- **Issue:** Initial evaluation of multiple embedding models proved more time-consuming than anticipated, with each model requiring significant computational resources for proper evaluation.
- **Impact:** Delayed Week 3 benchmarks by 1-2 days. Required adjustment of evaluation timeline.
- **Resolution:** Focused evaluation on two most promising models (BGE and E5) based on literature review and preliminary tests. This allowed for deeper, more comprehensive evaluation of selected models rather than shallow evaluation of many models.
- **Status: Resolved** - BGE-small and E5-base selected with comprehensive benchmarks documenting trade-offs.

3.2 PDF Text Extraction Quality

- **Issue:** Complex PDFs with tables, multi-column layouts, and embedded images caused text extraction errors. Initial implementation using only PyPDF2 resulted in incomplete text extraction.
- **Impact:** Some documents had incomplete text extraction, affecting retrieval quality for those documents.
- **Resolution:** Implemented dual extraction strategy using PyPDF2 and pdfplumber with intelligent fallback. Text from both extractors is merged, with pdfplumber used as fallback for problematic sections.
- **Status: Resolved** - Improved extraction accuracy to 95%+ on test document set.

3.3 LLM API Rate Limiting

- **Issue:** Google Gemini API rate limits affected development and testing speed, particularly during intensive testing periods.
- **Impact:** Slower iteration during Week 4-5 development. Some tests had to be delayed or run during off-peak hours.
- **Resolution:** Implemented response caching, retry logic with exponential backoff, and request batching. Optimized prompt design to reduce token usage.
- **Status: Mitigated** - Working within API limits with optimized requests and caching strategy.

3.4 Frontend-Backend Integration

- **Issue:** CORS configuration and async streaming response handling presented integration challenges. Initial attempts at streaming responses failed with CORS errors.
- **Impact:** Delayed real-time chat functionality by 1 day. Required deeper investigation of FastAPI CORS middleware configuration.
- **Resolution:** Properly configured CORS middleware with specific origins, methods, and headers. Implemented proper async handling for streaming responses using Server-Sent Events.
- **Status: Resolved** - Full integration working with streaming support for real-time chat.

3.5 Context Window Management

- **Issue:** Long documents exceeded LLM context window in some cases, causing truncated responses or inability to use all retrieved chunks.
- **Impact:** Some queries on long documents couldn't utilize all relevant context, potentially affecting answer quality.
- **Resolution:** Implemented smarter chunking with configurable overlap and context compression. Added relevance filtering to prioritize most important chunks.
- **Status: Partially Resolved** - Will be enhanced in Week 6-7 with advanced features including context compression and re-ranking.

3.6 Ongoing Concerns

Several challenges continue to be monitored and will be addressed in upcoming weeks. These include:

- **Retrieval Accuracy:** Will be improved in Week 6-7 with re-ranking strategies and hybrid search combining dense and sparse retrieval methods. Expected improvement of 10-15% in MRR metrics.
- **Response Quality:** Will be enhanced with optimized prompt engineering and better context utilization to be implemented in Week 6-7.
- **Performance Optimization:** Comprehensive performance profiling and optimization will be conducted in Week 8-9, focusing on latency reduction and resource efficiency.
- **Scalability:** Load testing and stress testing will be performed in Week 8-9 to ensure system can handle concurrent users and large document collections.

4. Next Steps

The following sections detail the planned work for the remainder of the project, organized into weekly sprints from June 25, 2026 to July 30, 2026. Each sprint builds upon the previous one, following the implementation plan established in the project proposal.

4.1 June 25 - July 8, 2026 (Week 6)

Hybrid Retrieval & Optimization

- Implement hybrid retrieval (dense + BM25) with fusion strategies
- Set up cross-encoder reranking models
- Create query expansion system with synonym generation
- Implement chunk optimization (size and overlap analysis)
- Build context compression algorithms
- Set up MS MARCO evaluation framework
- Set up SQuAD v2 evaluation framework
- Implement Recall@K and MRR metrics
- Create performance comparison tables
- Build UI for query input and results display
- Write comprehensive tests for all components
- Prepare IEEE-style report sections
- Complete documentation for hybrid retrieval system

4.2 July 9 - July 15, 2026 (Week 7)

Optimization & Evaluation - Part 1

- Performance profiling and bottleneck identification
- Implement caching strategies for frequently accessed data
- Optimize FAISS index for faster retrieval
- Implement request batching and parallelization
- Create comprehensive evaluation benchmark suite
- Implement A/B testing framework for features
- Add performance monitoring and metrics collection
- Optimize memory usage and resource allocation

4.3 July 16 - July 22, 2026 (Week 8)

Optimization & Evaluation - Part 2

- Complete performance optimization based on profiling
- Implement advanced evaluation metrics (NDCG, diversity)
- Create automated performance regression testing
- Optimize prompt engineering for cost efficiency
- Implement query result caching
- Load testing and stress testing
- Create performance dashboards and reports
- Document optimization strategies and results

4.4 July 23 - July 30, 2026 (Week 9)

Refactoring & Documentation

- Code refactoring for improved maintainability
- Implement design patterns where appropriate
- Improve error handling and logging
- Create comprehensive API documentation
- Write user guides for all features
- Create developer documentation
- Implement configuration management system
- Add comprehensive inline documentation
- Final system polish and deployment preparation

4.5 Additional Planned Sprints

Following Week 9, the project will continue with final weeks:

- **Week 10 (July 31 - August 6, 2026):** Final refactoring, documentation completion, and code quality assurance. Focus on polish and preparation for final delivery.
- **Week 11 (August 7 - August 13, 2026):** Final demo preparation, system polish, deployment configuration, and final presentation preparation. All deliverables finalized and packaged.

4.6 Timeline Summary

Week	Dates	Focus Area	Key Deliverables
Week 6	Jun 25 - Jul 8	Hybrid Retrieval	Dense+BM25, Reranking, Chunk Optimization
Week 7	Jul 9 - Jul 15	Optimization	Profiling, Caching, Fine-tuning
Week 8	Jul 16 - Jul 22	Evaluation	MS MARCO, SQuAD v2, Metrics
Week 9	Jul 23 - Jul 30	Analysis	Failed Cases, Results Analysis
Week 10	Jul 31 - Aug 6	Final Docs	IEEE Report, Presentation
Week 11	Aug 7 - Aug 13	Final Demo	Presentation, Deployment

4.7 Expected Outcomes

By the end of Week 11, the project will deliver a fully functional RAG-based question answering system with advanced features including re-ranking, hybrid search, multi-document QA, context window management, and query rewriting/expansion. The system will be optimized for performance, thoroughly tested, well-documented, and ready for deployment. All objectives outlined in the project proposal will be achieved.